

Language Modeling without Generation: Decoding a System One Model

Adhyaay Karnwal
Independent

github.com/adhyaay-karnwal/jev-chat

September 2026

Abstract

Jev, TypeSafe’s first System One model, is trained with reinforcement learning for calibrated decisions (RLCD). It returns typed Choice, Score, and Noul answers with probabilities. It does not emit tokens. This paper asks whether that interface can still implement a chatbot, by treating generation as a sequence of decisions over a finite codebook.

It cannot, not as an autoregressive language model. On eight short prompts run live against `jev-latest`, naive unit decoding produces locally plausible pieces that do not compose: Hello. Hello. Hello... and The short answer is 1 for 1+1 are decoder failures, not UI glitches. Greedy decoding with a closed codebook and a termination gate recovers short answers (automatic match 8/8) but remains ungrammatical on open prompts. Selecting a complete reply in one Choice—the use Jev was designed for—uses $39\times$ fewer input tokens and $6\times$ less wall time than naive autoregression, and is the only strategy that is consistently grammatical.

The result is not that Jev is a poor model. It is that RLCD was not a language-modeling objective, question independence forbids left-context composition, and a 255-way Choice is a classifier, not a vocabulary. System One models should select, not speak.

1 The question

Decoder-only transformers generate text by sampling $P(x_t \mid x_{<t})$ [2, 11, 12, 14]. Instruction following was then trained with RLHF [3, 9]. TypeSafe starts from the opposite contract [1, 13]: the model must not write. It must decide.

That is a serious claim. If it is true, most “LLM-in-the-loop” software is paying a chatbot tax—serial tokens, parseable JSON, uncalibrated confidence—for a judgment that could have been a typed distribution. If it is false, System One is a classifier API with a new name. This paper does not re-run TypeSafe’s workflow evals. It asks a narrower, more hostile question: can you invert the interface and get language out?

We implement four decoders on the public Jev API, including the one that produced the looping greetings in our demo, and measure them on a live eight-prompt suite. The suite is small on purpose. The failures are qualitative and cheap to exhibit; inflating n would not make an ungrammatical decoder a language model.

2 Jev, in the terms it was trained

2.1 System One

Kahneman distinguished fast, associative System 1 from slow, deliberative System 2 [6]. TypeSafe borrows the name for a model class: unstructured state in, typed questions out, in one parallel pass [13]. The analogy is the latency contract, not the error profile. Jev is named for William Stanley Jevons [5]: cheaper intelligence should raise, not lower, the volume of decisions software is willing to ask.

Three primitives cover the output space:

- Choice — categorical distribution over ≤ 255 named options, plus a concentration statistic called confidence.
- Score — distribution over an ordered rubric; the returned score is the probability-weighted index.

- Noul — $P(\text{yes}) \in [0, 1]$ for a predicate. No separate confidence.

Questions in one request share state, are scored independently, and do not condition on one another. Adding questions is almost free in wall-clock and linear in tokens [13]. That independence is the feature for software (no hidden coupling) and the obstacle for language (no $x_t \mid x_{t-1}$).

2.2 RLCD

TypeSafe contrasts three post-training paths [1]. RLHF optimizes human preference and, as a side effect, drops modes [9]. RLVR optimizes verifiable rewards and buys reasoning with serial test-time compute. RLCD optimizes calibrated decisions: a reported 0.8 should be right about 80% of the time, in aggregate [4]. The model is forbidden from generating strings, so it cannot type-error its way around a schema. Hallucination, in the usual sense of inventing a token outside V , is undefined: V is the caller’s criteria.

Two numbers from the launch materials, which we treat as vendor-reported rather than reproduced: about \$0.042 per million input tokens, output tokens unmetered; end-to-end latency 70–500 ms on System One workloads [1]. Our own calls sat in that latency band. We do not claim TypeSafe’s workflow-eval speedups (up to 194× vs. frontier LLMs). Those workloads are not generation.

2.3 What Jev is not

Jev is not a small LLM with a JSON grammar. Grammars constrain decoding of a generative model. Jev has no generative head to constrain. It is also not, on the public evidence, a drop-in replacement for next-token prediction. The rest of this paper is the measurement of that gap.

3 Method

Let C be the conversation and $y_{<t}$ the assistant prefix. A codebook $\mathcal{U}(C, y_{<t})$ of size ≤ 200 is built in code: EOS, discourse phrases, copy spans from C [10], function words, digits, frequent content words, and other. Jev returns

$$p = \text{Choice}(\mathcal{U}(C, y_{<t}) \mid C, y_{<t}).$$

We sample $u_t \sim p$ (temperature T , nucleus p_{top}) and set $y_t = y_{<t} \cdot \text{surface}(u_t)$. A companion Noul asks whether $y_{<t}$ is already a complete reply. If other wins, a second request factors the tail as $P(\text{letter}) P(\text{word} \mid \text{letter})$, the hierarchical softmax move [8] pointed at a 6,000-word list. Speculative fan-out [7] asks $K=6$ “if the prefix were extended by u ” follow-ups in the same call; independence means the premise must be written into the instructions, not the state.

This is S1LM. Four strategies:

Naive AR. $T=0.7$. After a completed sentence the codebook re-offers greeting phrases, including Hello. Termination Noul must exceed 0.72. This is the decoder that shipped in the first demo.

Constrained AR. $T=0.3$. Used labels are banned. Completed sentences do not re-open the greeting list. Termination threshold drops to 0.40 once y ends in .?!.

Greedy AR. Constrained codebook, $T=0$, no speculative follow-up.

Select. One Choice over complete candidate replies (greetings, yes/no, digits, a short factual list including Paris and Tuesday, copy unigrams). One call. No prefix.

Select is not a clever LM. It is TypeSafe’s documented pattern—select instead of generate—applied to chat.

4 Experiments

All runs hit jev-latest on 16 September 2026 from a single client. Eight prompts: a greeting, 1+1, 2+2, capital of France, a yes/no, a primary color, the day after Monday, and “spell cat.” Automatic match is a conservative regex (contains the expected atom; greeting must begin with hello and contain it once). Author fluency is a binary judgment: would one send this string as a reply. $n=8$ is a probe, not a leaderboard.

Automatic accuracy ranks greedy AR first (8/8). That ranking is misleading. the is paris and is tuesday match the regex and fail as English. Judged as

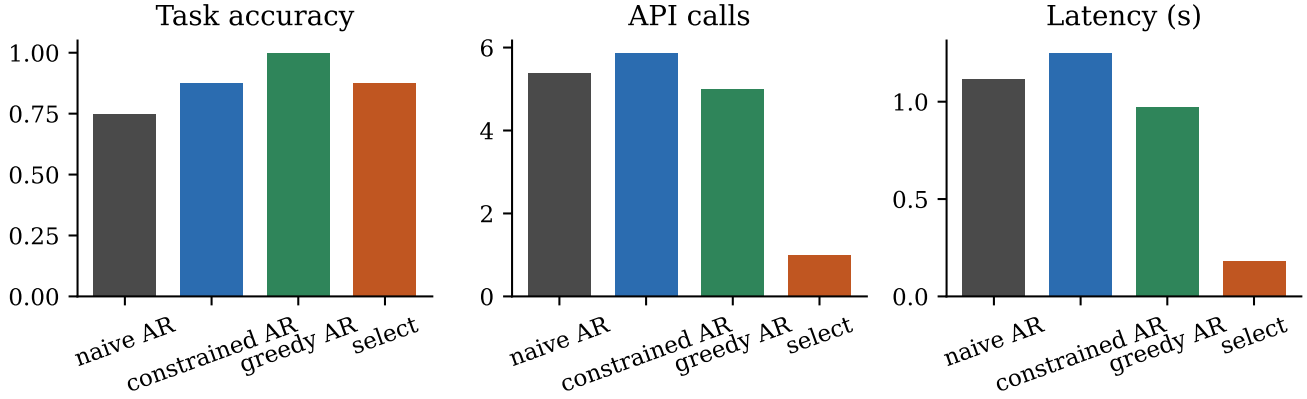


Figure 1: Automatic accuracy, API calls, and latency. Select is a different algorithm, not a faster AR decoder.

Table 1: Live Jev, eight prompts. Tokens are input tokens (output is unmetered). Fluency is author-judged grammaticality.

Strategy	Acc.	Fluency	Calls	Seconds	kTok
Naive AR	0.75	0.13	5.4	1.12	43.2
Constrained AR	0.88	0.50	5.9	1.25	44.3
Greedy AR	1.00	0.63	5.0	0.97	21.9
Select	0.88	1.00	1.0	0.18	1.1

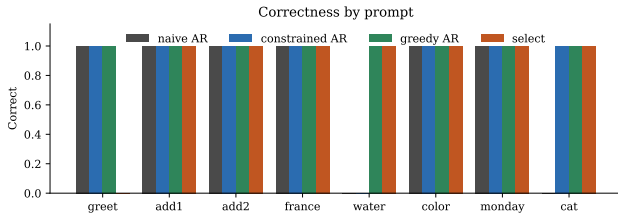


Figure 2: Automatic correctness by prompt. Greedy AR matches every regex; several of those strings are still not English.

sentences, the ranking flips: select 8/8, greedy 5/8, constrained 4/8, naive 1/8 (the lone fluent naive output is 4). Select’s only automatic miss is the greeting, where greedy Hello. loses to OK.—a complete, well-formed, wrong speech act. Coverage, not calibration, caused that miss: both strings were in the candidate set and the mode was OK.

Token traffic is the other axis. Naive AR sends a ~ 200 -way Choice plus speculative follow-ups on every step (mean 43,189 input tokens / prompt). Select sends one short list (mean 1,099). At the published \$0.042/MTok, that is about \$0.014 versus \$0.00037 per eight-prompt pass—not counting the AR tail calls.

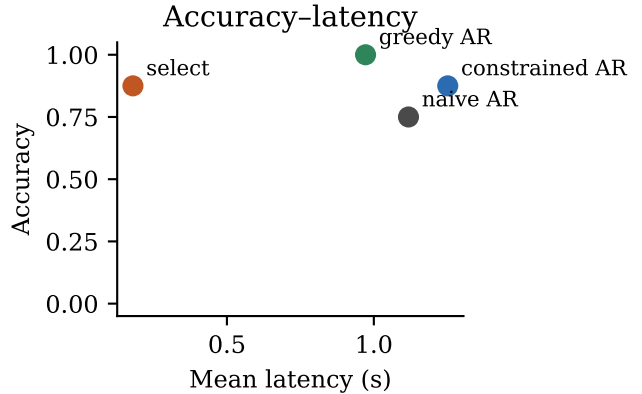


Figure 3: Accuracy against latency. The attractive point is select, not a better sampler on the AR loop.

5 Why the demo looked broken

The first public UI used naive AR. Two traces from that session:

User: Hello, this is a test
Jev-chat: Hello. Hello. Hello. Hello. ...

User: What is 1+1
Jev-chat: The short answer is 1

Neither is mysterious.

Collapse on Hello. After a completed sentence the naive codebook re-inserts the opener list. Hello. is always a legal next unit. The Noul $P(\text{done})$ after the first Hello. reached 0.59 in the logged naive greeting run, then fell back to 0.02 and never crossed 0.72 (Figure 4). The decoder hit max_calls. This is the GAN mode-collapse picture TypeSafe draws for RLHF [1], reconstructed in our codebook: a single

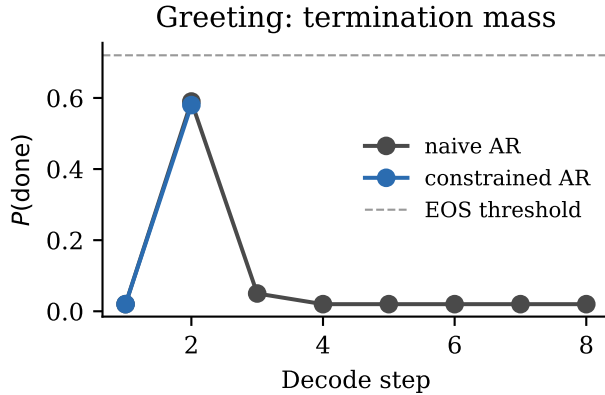


Figure 4: Termination Noul on the greeting prompt. Naive AR never clears 0.72. Constrained AR stops on the second call.

Table 2: Raw strings, greedy AR vs. select. Italics: automatic match with broken syntax.

Greedy AR	Select
Hello.	OK.
2	2
4	4
the capital is paris	Paris
Yes.	Yes.
Sure. a primary color such as red	red
the is tuesday	Tuesday
Sure. the word cat is a three letters	cat

attractive option is re-offered, and the termination bit is not calibrated for this out-of-distribution use.

Constrained AR stops. $P(\text{done})=0.58$ exceeds the 0.40 complete-sentence gate; the opener list is not re-offered; the reply is Hello.

$1+1 \rightarrow 1$. The short answer is an opener. 1 is a digit in the closed set, and a unigram in the user string. Under sampling, the model can pick the copied operand instead of the sum. Constrained and greedy AR, and select, all returned 2 on the same prompt in the controlled run. The screenshot is a real sample from a real decoder, not a different model.

6 What the pieces are doing

Independence is doing most of the damage. A Choice over next words given $y_{<t}$ is scored as an isolated judgment. It does not have to be a grammatical continuation of the last three words, only a plau-

sible unit given the conversation as a blob. That is why greedy AR can emit the capital is paris—four locally defensible decisions—and also the is tuesday. There is no likelihood of the string.

The codebook is doing the rest. A 255-way Choice cannot hold English. Hierarchical letter→word recovery is how paris appears at all (rank 138 in the p -bucket of a 6,000-word list; it is not in the primary 80 content words). Copy and phrase inventories leak question text into the reply (a primary color such as red). Speculative follow-ups save a round trip when they fire; they cannot create dependence the API does not have.

Select sidesteps both problems by moving composition into code. The model is asked the question it was trained to answer: which of these complete objects. When the object is in the list, Jev is fast, cheap, and grammatical. When it is not, select cannot invent it. That is the actual System One limitation, and it is the honest one.

7 Discussion

A frontier decision model can look like a language model for a few steps and then fall apart. The automatic metric in Table 1 would have hidden that. Any later paper on “Jev as an LLM” should report a fluency or human preference number, not a contains-the-answer regex.

The productive use of Jev in a dialogue system is not S1LM. It is a planner: speech act, whether to answer, whether the codebook contains the answer, whether to stop. Text should come from a generator, a template, or a select over retrieved replies. Mixing those—Jev routes, something else writes—is the architecture TypeSafe argues for [13]. We failed that test in the first demo by asking Jev to write.

Could a System One model be trained as a language model? Only with a different objective: sequential dependence, a large vocabulary, and a termination bit that is calibrated on prefixes, not documents. That would be a new model, not a new decoder on Jev.

8 Limitations

Eight prompts, one seed per cell, one API snapshot. No human raters beyond the author. No compari-

son to a grammar-constrained LLM of similar cost. The select candidate list contains Paris and Tuesday by construction; that is coverage engineering, and it is also the point. Vendor latency and pricing are taken from public docs except where we measured our own calls. Jev is early-access; the numbers will move.

9 Conclusion

Jev is a calibrated decision model. We built an autoregressive decoder around it anyway. The decoder can retrieve short facts from a closed set. It does not speak. The screenshot failures are the method working as specified: independent questions, a re-offered greeting, an uncalibrated stop bit, a copied digit. One-shot selection over complete replies is faster, cheaper, and grammatical, because it is the task RLCD was trained for.

Code, traces, and this paper: <https://github.com/adhyaay-karnwal/jev-chat>.

References

- [1] Diogo Almeida. Introducing system one models and jev. TypeSafe AI, September 2026. URL <https://typesafe.ai/blog/introducing-system-one-models-and-jev>.
- [2] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33:1877–1901, 2020.
- [3] Paul F. Christiano, Jan Leike, Tom B. Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- [4] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. *Proceedings of the 34th International Conference on Machine Learning*, pages 1321–1330, 2017.
- [5] William Stanley Jevons. *The Coal Question*. Macmillan, London, 1865.
- [6] Daniel Kahneman. *Thinking, Fast and Slow*. Farrar, Straus and Giroux, New York, 2011.
- [7] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning*, pages 19274–19286, 2023.
- [8] Frederic Morin and Yoshua Bengio. Hierarchical probabilistic neural network language model. In *Proceedings of the Tenth International Workshop on Artificial Intelligence and Statistics*, pages 246–252, 2005.
- [9] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, volume 35, pages 27730–27744, 2022.
- [10] Abigail See, Peter J. Liu, and Christopher D. Manning. Get to the point: Summarization with pointer-generator networks. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics*, pages 1073–1083, 2017.
- [11] Claude E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3):379–423, 1948.
- [12] Ilya Sutskever, Oriol Vinyals, and Quoc V. Le. Sequence to sequence learning with neural networks. In *Advances in Neural Information Processing Systems*, volume 27, 2014.
- [13] TypeSafe AI. Typesafe documentation, 2026. URL <https://docs.typesafe.ai/>.

- [14] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 2017.